PRODUCT IDEA PROPOSAL

SQL Readiness

A free, self-hosted scanner that scores how ready a SQL Server / Azure SQL database is for text-to-SQL and natural-language BI — and hands back a prioritized, runnable plan to fix it.

PREPARED BY	CATEGORY	STATUS
Naveen Naik	**Enterprise data tooling**	**Working prototype**

> *“The readiness score measures whether you should expect good text-to-SQL. The remediation loop makes it true. The planned eval layer proves it.”*

1 Executive summary

Text-to-SQL and LLM analytics are only as good as the schema beneath them. When a model is pointed at a database of cryptic table names, undocumented columns, and undeclared foreign keys, it guesses — and the failure looks like a model problem when it is really a **schema-readiness** problem.

SQL Readiness turns a fuzzy question — “is our database ready for text-to-SQL?” — into a single, defensible **0–100 score** backed by concrete, runnable findings. A read-only agent runs inside the customer’s network; nothing but metadata ever leaves. Teams review an executive-readable report, fix the gaps through a guided remediation loop, and re-scan to prove the lift — all without exposing a single row of data.

- **Deterministic & transparent.** Every point traces to a finding; no black box.
- **Privacy-first.** Read-only, metadata-only, air-gap friendly — the trust story is the product.
- **Actionable.** Findings come with copy-pasteable fixes and an agent that applies approved changes.
- **Measurable.** Repeat scans track the score over time; a planned eval layer proves query-accuracy gains.

2 The problem

Enterprises are racing to put natural-language interfaces on their databases, but the schemas were built for applications and analysts, not language models. The result is unreliable answers that erode trust in the whole initiative.

- Cryptic names (`tbl_cust`, `dt1`) give the model nothing to reason from.
- Undocumented tables and columns strip away the semantic context an LLM needs.
- Undeclared foreign keys hide the join paths behind real business questions.
- Duplicated / ambiguous tables make the model pick the wrong source.
- There is no accepted, objective way to say whether a database is “ready” — or to justify the cleanup work.

3 The solution

SQL Readiness measures schema readiness *before* an LLM is pointed at the database, and drives the work to close the gap. It is delivered as three decoupled parts joined by simple JSON contracts.

The loop

4 The scoring model

The score is computed by a scoring engine that is fully decoupled from collection, so the rubric can evolve and historical scans can be re-scored. Nine dimensions carry relative weights; each is

importance-weighted per table by usage and row count, so documenting a hot fact table moves the score more than a dead archive.

Dimension	Weight	What it rewards
Referential integrity	20	Declared foreign keys / join paths
Semantic naming clarity	15	Human-legible table & column names
Documentation coverage	15	Descriptions on tables and columns
Naming consistency	10	Uniform conventions across the schema
Data type sanity	10	Appropriate, non-overloaded types
Ambiguity / duplication	10	One obvious source per concept
View / proc-obscured logic	8	Business logic that isn't hidden
Cardinality signal	5	Distinguishable dimensions vs facts
Junk / placeholder data	4	Clean, non-test content

The weights are **relative, not percentages** — they sum to 97 by design. The overall score normalizes across whichever dimensions apply (Σ(dimension score × weight) / Σ(weight)), so only the ratios matter. PII exposure and query-performance risk are reported as **separate flags**, never folded into the readiness score.

5 Architecture & the trust boundary

Three components, joined by JSON contracts, keep collection, scoring, and presentation independent — and keep customer data inside the customer network.

Scan agent (CLI)

C# / .NET 9. The only component that touches the database, under a least-privilege read-only login. Emits findings JSON, can render a self-contained HTML report for air-gapped sites, and is the sole write path.

Scoring engine

Consumes findings JSON, applies severity deductions and dimension weights, produces the score and rollups. Decoupled so the rubric can change and old scans re-score.

Report web app

Next.js. Reports, score trends, inline remediation with live score preview, plan export, multi-organization isolation, and a JSON API. Never connects to the database.

The trust boundary is the product.
Only findings — metadata and derived statistics — ever leave the customer network; raw row data never does. The web app never reaches into the database, and remediation is applied by the agent inside the network from a plan the customer reviewed — never by a cloud service reaching in.

6 Key capabilities shipped

Remediation write-back

Fixing documentation is a first-class loop that respects the trust boundary end to end. Owners edit descriptions inline in the report and watch the projected score update live — computed by the same

scoring engine, not a faked percentage. Exporting a `remediation-plan.json` yields a decoupled artifact the agent applies inside the network: idempotent (resolves targets by name), atomic (one transaction — a mid-plan failure writes nothing), and guarded (refuses to write unless the plan's target database matches the connection). A dry-run prints the exact T-SQL first.

Automatic findings upload (org-scoped ingest tokens)

The scan runs entirely offline; uploading is a separate, opt-in step. An organization owner mints an **ingest-only** token (scoped to one org, hashed at rest, shown once, revocable instantly). The agent then pushes findings with `--upload` (scan then send) or `--push` (send a file carried out of an air-gapped environment). The destination org is derived from the token, an audit log records every upload, and tokens rotate with zero downtime.

7 Roadmap

The score measures schema quality (the input to text-to-SQL). The next layers measure and package the outcome — behind the same trust boundary.

BYOM model layer

A bring-your-own-model abstraction so AI features call a model the customer chose — local (Ollama / vLLM) by default, cloud only on explicit, previewed opt-in. It powers schema-aware description enrichment, and is where a documented column literally becomes a better prompt.

Eval harness — "prove it"

Measure text-to-SQL **execution accuracy** before and after remediation, turning "ready for text-to-SQL" into "proven to improve it" — the enterprise sales story.

Packaging & distribution

Signed, self-contained agent binaries (no runtime needed on the target), a Windows MSI for Group Policy deployment, and a container image — so a privacy-conscious enterprise can adopt it with zero friction.

8 Who it's for, and why it wins

Target users

- Data platform engineers & DBAs preparing databases for LLM analytics
- Analytics leads justifying remediation work to a team
- Platform teams gating readiness in CI/CD

Why it wins

- Objective, deterministic number where none existed
- Privacy posture enterprises can actually approve
- Not just a report — a remediation loop that closes the gap
- A path from "ready" to "proven" via the eval layer

9 Current status

SQL Readiness is a working prototype. The scan agent, the deterministic scoring engine (with cross-language parity tests), the themed web report with score trends and multi-org isolation, the full remediation write-back loop, and the org-scoped automatic-upload pipeline (tokens, endpoint, agent, audit log, rotation) are all built and tested. The BYOM model layer, the eval harness, and signed distribution artifacts are specified and scheduled.

SQL Readiness — text-to-SQL readiness for SQL Server & Azure SQL. Read-only, metadata-only, nothing leaves the customer network.
Prepared by Naveen Naik.